Scripting Languages - Lecture 02

Lecture overview

Lecture 02 introduces fundamental scripting concepts in Python, focusing on values, variables, strings, and lists. The session begins by treating Python as a calculator, where operators act on operands to form expressions governed by standard mathematical precedence, which can be overridden using parentheses. Numeric data is categorized into integers for whole numbers and floating-point numbers for decimals, with distinct behaviors for operations like division versus floor division. Variables serve as named containers for storing and reusing values, allowing for sequential updates through assignment or shorthand operators like +=. The lecture emphasizes breaking complex calculations into intermediate variables to improve code clarity, as demonstrated in a reading time estimation example.

The second half of the lecture distinguishes between data types, specifically strings (text) and lists (general sequences), highlighting that operators behave differently depending on the type; for instance, the plus operator performs arithmetic addition on numbers but concatenation on strings. Strict type compatibility is enforced, requiring explicit conversion functions like `str()` and `int()` to mix numeric and textual data. A critical distinction is drawn between immutable objects, such as strings which cannot be modified after creation, and mutable objects like lists, which allow direct item changes, additions via `append()`, and removals via `remove()` or `pop()`. The session concludes by covering sequence operations including indexing, slicing, length calculation, sorting methods, and the structure of nested lists, while noting that comments using the hash symbol are used to annotate code without affecting execution.

Slide 1: faculty of arts

This lecture introduces the second session on scripting languages within the Faculty of Arts, presented by Tim Van de Cruys on September 30, 2026. The primary topics covered in this session are values, variables, strings, and lists.

Slide 2: Python as a calculator

Python functions as a calculator by utilizing operators, which are reserved symbols representing mathematical computations applied to values called operands. Standard operations include addition (+), subtraction (-), multiplication (*), division (/), and exponentiation (**). Additionally, Python supports modulus (%) to find the remainder of a division, such as 100 % 8 equaling 4, and floor division (//) to return only the whole number part of a quotient.

Slide 3: Python as a calculator

Python functions as a calculator where one or more operators form an expression. The example `1 + 2 * 3` evaluates to 7, demonstrating standard operator precedence. However, using parentheses in `(1 + 2) * 3` changes the evaluation order to yield 9, illustrating how grouping alters calculation results.

Slide 4: Python as a calculator

Operators act on operands to form expressions in Python, which follows standard mathematical precedence rules where exponentiation occurs before multiplication/division, and those before addition/subtraction. For example, `1 + 2 * 3` evaluates to 7 because multiplication is performed first, whereas `(1 + 2) * 3` yields 9 by using parentheses to explicitly override the default order of operations.

Slide 5: Numeric data types

Numeric values in scripting languages are categorized into distinct types: integers for whole numbers (e.g., 1, -2) and floating-point numbers (floats) for decimals (e.g., 2.5). Integers typically require less computational time to represent than floating-point numbers, a performance difference students should recognize. These models distinguish between exact whole numbers and approximate decimal representations based on their specific data type requirements.

Slide 6: Numeric data types

Numeric values in scripting languages possess specific types: integers represent whole numbers, while floating-point numbers include decimals. Standard division (e.g., `5 / 2`) returns a float like 2.5, whereas floor division (`5 // 2`) yields an integer. These distinct types exist primarily due to implementation efficiency under the hood, though this technical detail is often less critical for general usage.

Slide 7: Variables

A variable is a name referring to a value, created or updated using the single equal sign (=) for assignment rather than equality. Variable names can be arbitrarily long and contain letters, numbers, and underscores, but they cannot start with a number and are case-sensitive. This mechanism allows intermediate results, such as `a = 5 / 2`, to be stored and reused later in the code.

Slide 8: Terminology

This slide defines three fundamental programming concepts: constants are fixed numerical values like 2 or 1.5; expressions are sequences combining operands (constants) and operators, such as "2 + 5"; and variables act as named containers holding values that can substitute for constants in computations. Variables, exemplified by names like 'a', 'b', or 'books', allow these stored values to be reused within further expressions.

Slide 9: Using variables

Variables can be used in expressions as substitutes for constants. The slide demonstrates this by evaluating the expression `a + 0.5`, which yields an output of `3.0`. This example illustrates how a variable's stored value is utilized during calculation within scripting languages.

Slide 10: Using variables

Variables can replace constants in expressions, such as using `a` to calculate `a + 0.5`. Programs execute sequentially line-by-line, meaning variables can be overwritten by new assignments; for instance, assigning `a = 2` replaces any previous value. While overwriting requires caution, it is essential for updating values, such as incrementing a counter when processing text files.

Slide 11: Updating variables

Updating variables is a common operation in scripting languages where a variable's value is increased by reassigning it. For example, if `a` is initially set to 2, executing the statement `a = a + 1` updates the variable to hold the new value of 3. This process demonstrates how a variable can be modified using its current value in an arithmetic expression.

Slide 12: Updating variables

Updating variables is a common operation where a variable's value is increased, such as assigning `a = a + 1` to change a from 2 to 3. A shorthand syntax exists for this operation using the += operator, written as `a += 1`, which produces the same result. Similar shorthand operators are available for other arithmetic operations, including subtraction (`-=`) and multiplication (`*=`).

Slide 13: An example: Reading Proust

This slide illustrates calculating the time to read Proust's In Search of Lost Time (1,267,069 words) at a rate of 238 words per minute for one hour daily, yielding approximately three months. While a single complex formula can produce this result, the professor emphasizes using intermediate variables to store values like "words per day" and "reading time in days." This approach improves code clarity and maintainability by breaking down the computation into manageable steps rather than relying on intricate bracketing.

Slide 14: Text

In Python, textual values are stored as strings (str), which represent a sequence of characters. Strings can be defined using single quotes, double quotes, or triple quotes depending on the content; for instance, double quotes are required if the text contains an internal single quote to prevent syntax errors. Triple quotes are specifically used for defining larger blocks of text, such as multi-line lyrics.

Slide 15: Types

A type is a class of values recognized by Python, such as numbers (int, float) or text (str), where every value always possesses a specific type. This type determines which operations are valid; for instance, mathematical operations apply to integers and floats but not to strings. Attempting incompatible operations, like adding an integer to a string, results in a TypeError because the language does not define such behavior.

Slide 16: Different behavior for types

In scripting languages, data types determine how operators function. For numeric types, the plus operator performs arithmetic addition (e.g., `3 + 3` results in 6). However, for string types, the same operator executes concatenation instead of addition, appending the second string to the first (e.g., `'3' + '3'` results in `'33'`).

Slide 17: Invalid operations for types

Attempting to add an integer and a string, such as `3 + '3'`, triggers a `TypeError` in Python because the operation is unsupported for these operand types. The resulting traceback explicitly identifies the incompatible types as `'int'` and `'str'`. This example illustrates that scripting languages enforce strict type compatibility for arithmetic operators, preventing implicit conversion between numbers and text.

Slide 18: Converting numbers to text and back

Python uses functions like `str()` and `int()` to convert between numbers and text, where `str(number)` transforms an integer into a string and `int(text)` reverses this process. A function is defined as a piece of code invoked by name that accepts input arguments within brackets and produces a return value. This notation, represented generally as f(x), applies the specific function to the provided value to achieve data type conversion.

Slide 19: Summary

This slide summarizes fundamental scripting language concepts, including values and types such as integers, floats, and strings. It illustrates expressions using arithmetic operations with parentheses, variable assignment, and type conversion functions like `str()` and `int()`. These elements collectively demonstrate the basic syntax for defining data and performing calculations in the language.

Slide 20: Comments

Comments are explanations added to code to improve readability, especially in large programs. In Python, comments begin with a hash symbol (#), and the interpreter ignores all text following it on that line. This feature allows developers to annotate their code without affecting program execution.

Slide 21: Comments

Comments are explanations added to large code blocks to improve readability by clarifying the programmer's intent. In Python, comments begin with a hash symbol (#), and the interpreter ignores all text following this character on that line. For instance, appending `#convert fahrenheit to celsius` after a temperature conversion formula explicitly documents the calculation for future readers.

Slide 22: Example: computing readability

The Automated Readability Index estimates text difficulty using a linear regression formula: 0.50(total words/total sentences) + 4.71(total characters/total words) − 21.43. This calculation yields a grade level score, where 1 represents kindergarten (age 5) and 14 corresponds to college students (age 18). The formula weights words per sentence at 0.5 and characters per word at 4.71 to determine the final readability metric.

Slide 23: Indexing: extracting one element

Indexing extracts a single element from a sequence using square brackets, where `a[n]` retrieves the n'th character. Counting starts at zero for positive indices, while negative indices count backward from the end (e.g., -1 is the last character). For example, in the string 'Marcel', `name[0]` returns 'M' and `name[-1]` returns 'l'.

Slide 24: Slicing: extracting a range of elements

Slicing extracts a range of elements using the syntax `a[n:m]`, where counting starts at zero and the result includes index `n` up to, but not including, index `m` (a half-open interval). If the start index `n` is omitted, extraction begins from the start; if the end index `m` is omitted, it continues to the end of the sequence. For example, with `name = 'Marcel'`, `name[1:3]` returns `'ar'`, while `name[:2]` and `name[1:]` return `'Ma'` and `'arcel'` respectively.

Slide 25: Indexing and slicing

Slide 25 introduces the topic of indexing and slicing within the context of scripting languages. The slide serves as a section header for lecture material 20/36, presented by Tim Van De Cruys at KU Leuven. No specific definitions, examples, or technical explanations regarding these operations are provided in the available source text or aligned transcript for this fragment.

Slide 26: Length

The `len()` function retrieves the number of characters in a string. For example, assigning the string 'Marcel' to the variable `name` and executing `len(name)` returns the integer 6, representing the total character count. This operation is fundamental for determining string size in scripting languages.

Slide 27: Functions

A function is a piece of code invoked by name that may take input arguments and either produce a result or an effect. Examples like `int()`, `print()`, and `len()` are built-in functions defined by the Python interpreter, using the notation $f(x)$ to apply function f to value x. While these predefined functions are available immediately, students will later learn to define their own custom functions.

Slide 28: Format strings

Slide 28 demonstrates two methods for combining text and variables in Python messages. The first approach uses string concatenation with the + operator, requiring explicit conversion of numeric variables to strings using `str()`. The second method utilizes the `print()` function's ability to accept multiple comma-separated arguments, which automatically handles spacing and type conversion without manual string casting.

Slide 29: Format strings

Format strings provide a clear method for combining text with variables, such as integers or floats, into readable messages. While manual string concatenation requires explicit type conversion and the standard print function handles conversion automatically, formatted strings using curly braces offer superior readability. This approach embeds variables directly within the string literal, eliminating cumbersome syntax while producing the same output.

Slide 30: Sequences

A sequence is an ordered collection of elements, with strings being a specific type containing characters. Basic operations include determining length, accessing items via indexing or slicing, concatenating sequences with +, and repeating them using `*`. While `*` performs mathematical multiplication on numbers, it repeats string elements when applied to text, and lists serve as more general sequences capable of holding any value type.

Slide 31: Lists

Lists are general sequences capable of containing any kind of value as items, including mixed types like numbers and text within a single list. They support indexing to extract or modify specific elements and can be nested, allowing lists to contain other lists for complex data structures. This flexibility makes lists a fundamental element used extensively in Python and most programming languages.

Slide 32: List indexing

Slide 32 introduces the topic of list indexing within the context of scripting languages. The slide serves as a section header for lecture material dated 26/36, presented by Tim Van de Cruys. No specific definitions, examples, or technical details regarding list indexing are provided in the available source text or aligned transcript for this fragment.

Slide 33: Changing items

Lists are mutable, allowing items to be changed directly as shown by updating `names[1]` from 'Mary' to 'Alice'. In contrast, strings are immutable, meaning characters cannot be modified after creation; attempting `name[0] = 'D'` raises a TypeError. Consequently, changing a string requires creating a new string object rather than modifying the existing one.

Slide 34: Immutable vs. mutable

Immutable objects, such as strings, cannot be changed after construction, which allows for more efficient implementation and quicker access. In contrast, mutable objects like lists permit changes to their size and content, making them more efficient when updates are required. This distinction in Python types dictates whether individual elements can be modified and influences how the objects are internally implemented.

Slide 35: Objects and their methods

In Python, every value is an object that encapsulates both data and the code used to manipulate it. A method is a function belonging to an object, where the object's type determines available operations. While regular functions use the syntax `function(argument)`, methods are invoked using dot notation as `obj.function(argument)` to apply specific operations to that object.

Slide 36: Adding items

The `append()` method adds an item to the end of an existing list by mutating it directly, whereas using the `+` operator combines two lists into a new list without altering the originals. For example, `names.append('Alice')` changes the `names` list in place, while `friends + enemies` creates a separate combined list. This distinction is critical: `append` modifies the original object, while concatenation generates a new one.

Slide 37: Removing items

The `remove(elem)` method finds and deletes the first occurrence of a specific value from a list without returning it. In contrast, the `pop(value)` method removes and returns the item at a specified index, while `pop()` with no arguments removes and returns the last item in the list. These operations modify the original list structure by permanently deleting the targeted elements.

Slide 38: Removing items (2)

Unlike methods like `remove` or `pop` that mutate a list, slicing creates a new list containing only a specific portion of the original data without altering the source. As demonstrated with `names[:2]`, this operation extracts elements starting from index 0 up to, but not including, the specified index, resulting in a new list like `['John', 'Mary']` while leaving the original `names` list intact.

Slide 39: Sorting

Sorting lists can be achieved using either the `sorted()` function or the `.sort()` method, which differ in how they handle the original data. The `sorted()` function returns a new sorted list while leaving the original list unchanged. Conversely, the `.sort()` method modifies the original list in place and does not return any value.

Slide 40: Nested lists: lists inside of lists

Nested lists contain lists as elements, such as `ages = [['John', 34], ['Mary', 19], ['Max', 62]]`. Accessing an element like `ages[0]` retrieves the inner list `['John', 34]`, while further indexing with `[1]` extracts specific values like `34`. Alternatively, individual elements can be accessed directly by chaining indices, such as selecting the first item of the second inner list to retrieve 'Max'.

Slide 41: Nested lists: lists inside of lists

Nested lists are structures containing lists within a list, such as `ages = [['John', 34], ['Mary', 19], ['Max', 62]]`. Elements are accessed using chained indexing: `ages[0]` retrieves the first inner list `['John', 34]`, while `ages[0][1]` directly accesses the specific element `34` by targeting the second item of that inner list.

Slide 42: Summary: strings vs lists

Strings and lists are both sequences with fixed order, length, and support for item or slice extraction. However, lists are more general because they can contain any data type, whereas strings consist exclusively of characters. Additionally, lists are mutable, allowing modification, while strings are immutable.

Slide 43: Practical session / reading

The first practical session utilizes a self-explanatory notebook where students should complete core exercises before attempting optional challenge tasks. Assistance is available today from 4pm to 6pm in room VHI 02.25 or online, while the lecture's code-along notebook on Toledo allows for re-running examples at home. Students are also required to read chapters 3 and 4 of Python for Everybody.

Key points to remember

The second half of the lecture distinguishes between data types, specifically strings (text) and lists (general sequences), highlighting that operators behave differently depending on the type; for instance, the plus operator performs arithmetic addition on numbers but concatenation on strings. Strict type compatibility is enforced, requiring explicit conversion functions like `str()` and `int()` to mix numeric and textual data. A critical distinction is drawn between immutable objects, such as strings which cannot be modified after creation, and mutable objects like lists, which allow direct item changes, additions via `append()`, and removals via `remove()` or `pop()`. The session concludes by covering sequence operations including indexing, slicing, length calculation, sorting methods, and the structure of nested lists, while noting that comments using the hash symbol are used to annotate code without affecting execution.